

Aula 9 - Versionamento e Git Avançado

Duração: 2 horas

Objetivos:

- Dominar estratégias de branching com Git Flow
- Automatizar workflows com GitHub Actions
- Realizar code reviews eficientes
- Aplicar boas práticas em repositórios colaborativos

Parte Teórica (40 min)

1. Git Flow: Estratégia de Branching

Diagram

Code

```
gitGraph
    commit
    branch develop
    checkout develop
    commit
    branch feature/login
    commit
    checkout develop
    merge feature/login
    branch release/v1.0
    commit
    checkout main
    merge release/v1.0
    branch hotfix/issue-123
    commit
    checkout main
    merge hotfix/issue-123
    checkout develop
    merge hotfix/issue-123
```

Branches principais:

- `main / master` : Código em produção
- `develop` : Integração de features
- `feature/*` : Novas funcionalidades

- `release/*` : Preparação para versão
- `hotfix/*` : Correções urgentes

2. GitHub Actions para CI/CD

Workflow típico:

1. Push para `feature/*` → Roda testes
2. Merge em `develop` → Gera build
3. Tag em `release/*` → Deploy em staging

3. Code Review Eficaz

Checklist:

- ✓ Funcionalidade implementada conforme requisitos
- ✓ Código legível e com boas práticas
- ✓ Testes unitários e cobertura adequada
- ✓ Documentação atualizada

Parte Prática (1h20min)

Atividade 1: Git Flow na Prática

Cenário: Adicionar feature de login

1. Crie e trabalhe em uma branch de feature:

```
bash
```

```
git checkout develop
git pull origin develop
git checkout -b feature/login
```

2. Simule commits:

```
bash
```

```
# Alterar arquivos
git add .
git commit -m "Implementa validação de senha"
```

3. Merge via Pull Request:

bash

```
git push origin feature/login  
# Criar PR no GitHub para merge em develop
```

Exercício:

- Crie uma branch `hotfix/header-null` a partir de `main`
- Faça um commit simulando uma correção

Atividade 2: GitHub Actions para Validação

1. Crie `.github/workflows/pr-validation.yml`:

yaml

```
name: PR Validation  
  
on: [pull_request]  
  
jobs:  
  build:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v3  
      - name: Set up JDK 17  
        uses: actions/setup-java@v3  
        with:  
          java-version: '17'  
      - name: Build and Test  
        run: mvn clean test
```

2. Configure branch protection:

- Settings → Branches → Add rule
- Exigir:
 - Status checks (build/tests)
 - Approvals (2 revisores)

Teste:

- Faça um PR com erro para ver a ação falhar

Atividade 3: Code Review Simulado

Cenário: Revisar PR #123 com código problemático

java

```
// Código para revisar (exemplo ruim)
public class Calculadora {
    public int divide(int a, int b) {
        return a / b; // Falta tratamento de divisão por zero!
    }
}
```

Boas práticas na revisão:

1. Comentários específicos (não apenas "não gostei")

❌ "Evite divisão sem tratamento de erro"

✅ "Sugiro adicionar `if (b == 0) throw new ArithmeticException()`"

2. Sugira alternativas com snippets:

java

```
public int divide(int a, int b) {
    if (b == 0) {
        throw new ArithmeticException("Divisor não pode ser zero");
    }
    return a / b;
}
```

Exercício em duplas:

- Um aluno cria um PR com código intencionalmente problemático
- Outro aluno revisa usando o modelo GitHub

Desafio Colaborativo (30 min)

Contexto:

1. Em grupo de 3:

- **Dev 1:** Cria branch `feature/notificacoes` e implementa (com erro)
- **Dev 2:** Abre PR e solicita revisão
- **Dev 3:** Revisa e pede alterações

2. Requisitos:

- Classe `NotificacaoService` com método `enviarEmail()`
- Validar formato do email (regex)
- Teste unitário para cenários válidos/inválidos

Critérios de sucesso:

- PR aprovado só após:
 - Todos os comentários resolvidos
 - Build passando
 - 100% cobertura para a nova classe

Encerramento

- **Discussão:** Qual parte do fluxo foi mais desafiadora?
- **Próxima aula:** Integração Contínua (CI/CD) Avançada

Tarefa:

- Configurar um workflow que gere Javadoc automaticamente
- Estudar Conventional Commits

Recursos:

- [Git Flow Cheatsheet](#)
- [GitHub Actions Marketplace](#)

Prontos para trabalhar como um time de verdade? Dúvidas? É só perguntar!  